- **Internship Name : Data Analysis**
- **Task Name: Task 4: Practice Reading and Writing .txt and .csv Files Using Python File Handling**
- **Your Name : Shubhangi Naresh Khonde**
- **Internship Organization :**
- **Date:**

1.Objective:

The objective of this task was to understand and practice file handling in Python. The task focused on creating, writing, reading, and appending data in text files using Python’s built-in open() function. It also covered reading and writing CSV files using Python’s built-in csv module.

2.Introduction:

File handling is an important concept in Python that allows programs to store, retrieve, and manage data from files. Python provides built-in functions and modules for working with different types of files.

In this task, .txt files were handled using the open(), write(), and read() functions. The with open() statement was also practiced for safer and automatic file handling.

For CSV files, Python's built-in csv module was used. The csv.writer(), csv.reader(), and csv.DictReader() functions were practiced to write, read, and process tabular data.

3.Concepts Covered:

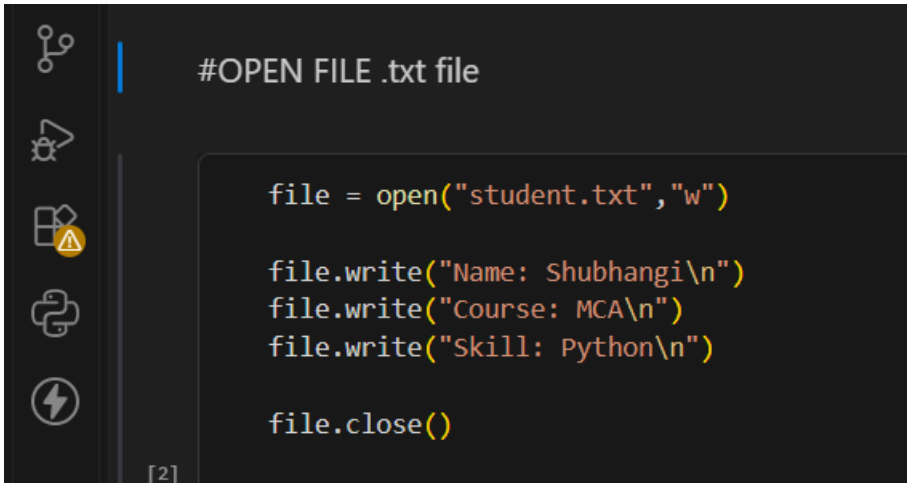
The following concepts were covered during this task:

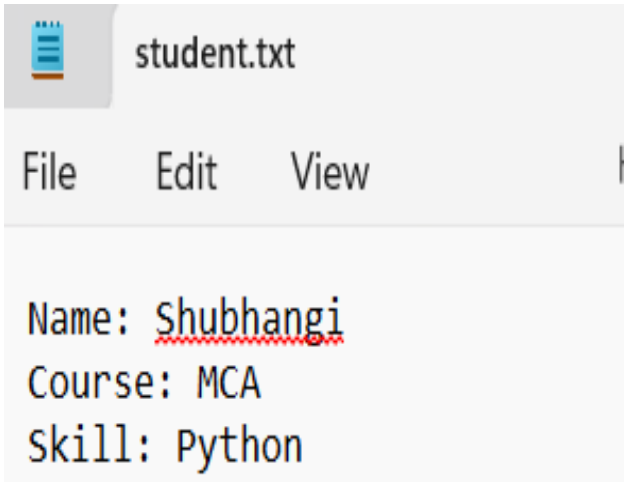
1. Python open() function
2. File modes:
    - "r" – Read
    - "w" – Write
    - "a" – Append
3. write() function
4. read() function
5. close() function
6. with open() statement
7. Python csv module
8. csv.writer()
9. writerow()
10. csv.reader()
11. csv.DictReader()
12. Reading specific CSV columns
13. Filtering CSV data using if conditions.

4.Code / Implementation:

Writing Data to a Text File:

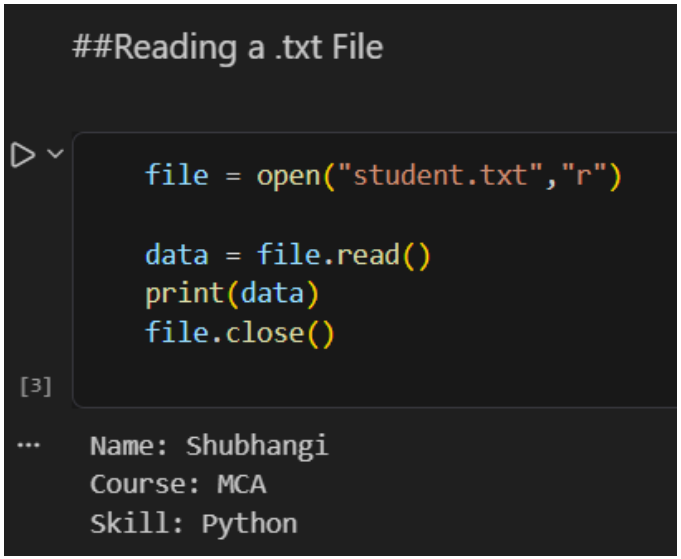
This code creates a text file named `student.txt` and writes student information into it.



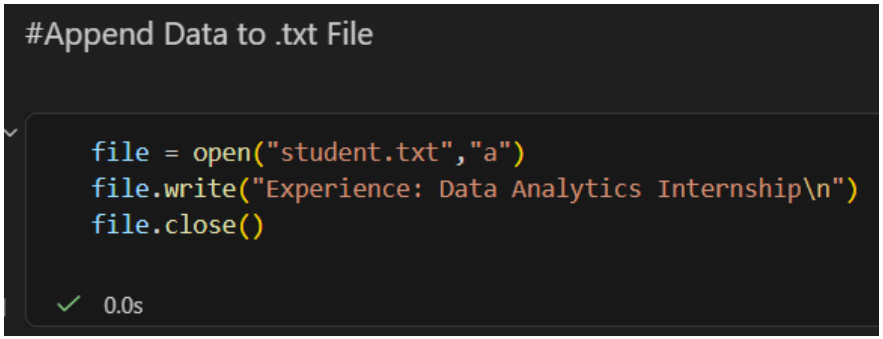


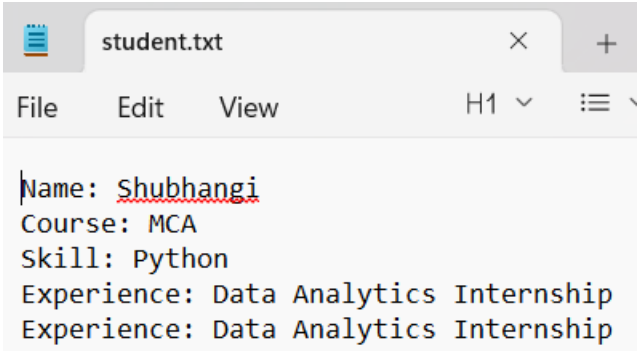
Reading Data from a Text File:

This code reads the complete content of the text file and displays it.



Appending Data to a Text File:





The append mode adds new data at the end of the existing file without removing the previous content.

Using with open():

The with open() statement automatically closes the file after the operation is completed.

```
#with Open file

with open("student.txt","r") as file:
    data = file.read()
print(data)
```

```
Name: Shubhangi
Course: MCA
Skill: Python
Experience: Data Analytics Internship
```

Creating and Writing to a CSV File

import csv

This code creates a CSV file and stores student information in rows and columns.

```
#Creating the CSV File

import csv
```

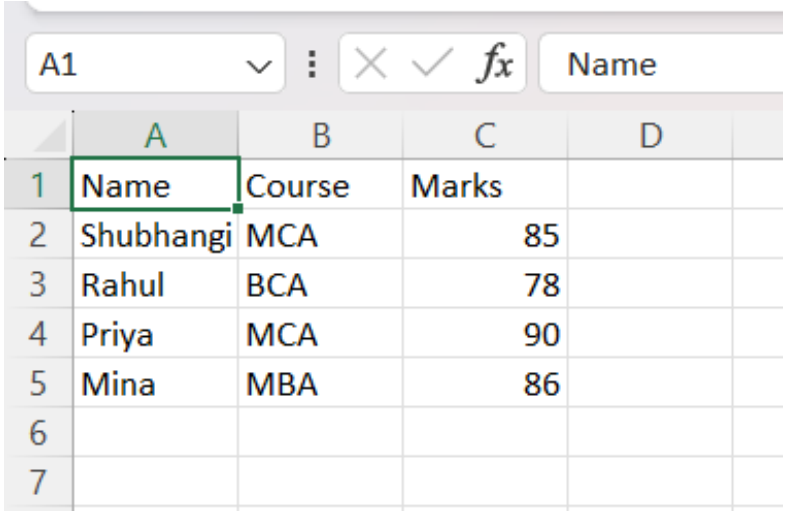
✓ 0.0s

```
import csv

with open("student.csv","w",newline="") as file:
    writer = csv.writer(file)

    writer.writerow(["Name","Course","Marks"])
    writer.writerow(["Shubhangi","MCA",85])
    writer.writerow(["Rahul","BCA",78])
    writer.writerow(["Priya","MCA",90])
    writer.writerow(["Mina","MBA",86])
```

✓ 0.0s



A1 | fx | Name

	A	B	C	D
1	Name	Course	Marks	
2	Shubhangi	MCA	85	
3	Rahul	BCA	78	
4	Priya	MCA	90	
5	Mina	MBA	86	
6				
7				

Reading a CSV File

This code reads each row from the CSV file and displays it.



Reading CSV Data Using DictReader

DictReader() reads each row as a dictionary using the CSV column names as keys.

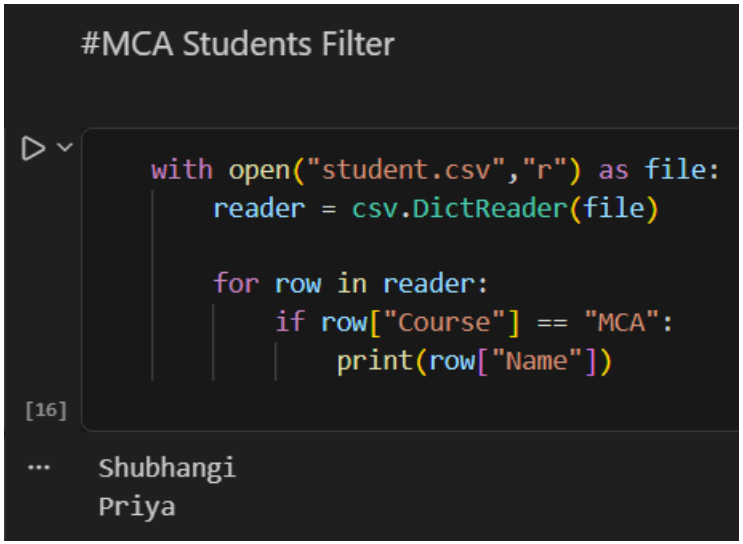
```
##CSV Data as Dictionary

    with open("student.csv","r") as file:
        reader = csv.DictReader(file)
        for row in reader:
            print(row)
            print(row["Name"])
```

```
{'Name': 'Shubhangi', 'Course': 'MCA', 'Marks': '85'}
Shubhangi
{'Name': 'Rahul', 'Course': 'BCA', 'Marks': '78'}
Rahul
{'Name': 'Priya', 'Course': 'MCA', 'Marks': '90'}
Priya
{'Name': 'Mina', 'Course': 'MBA', 'Marks': '86'}
Mina
```

Filtering CSV Data

This code displays only the names of students whose course is MCA.



Appending a New Row to a CSV File

This code adds a new student record to the existing CSV file.

```
#Existing In CSV New Row Add

    with open("student.csv","a",newline="") as file:
        writer = csv.writer(file)

        writer.writerow(["Amit","BCA",82])
        writer.writerow(["Amit","BCA",87])
```

5.Results:

The following results were successfully achieved:

- A .txt file named student.txt was created successfully.
- Data was written into the text file using write mode.
- Data was successfully read using read mode.
- New information was added using append mode.
- The with open() statement was successfully used for file handling.
- A students.csv file was created successfully.
- Student data was written into the CSV file using csv.writer().
- CSV data was successfully read using csv.reader().

- CSV records were accessed as dictionaries using csv.DictReader().
- Specific columns were accessed using column names.
- MCA students were filtered using an if condition.
- A new student record was appended to the existing CSV file.

6.Key Learning:

Through this task, I learned how to work with external files using Python. I understood the difference between read, write, and append modes. I also learned how to use the with open() statement for safe file handling.

Additionally, I learned how to use Python's built-in csv module to create, read, write, and update CSV files. I understood how CSV data can be processed row by row and how DictReader() makes it easier to access data using column names.

These file-handling concepts are useful for data analysis because real-world datasets are often stored in formats such as CSV and text files.

7.Conclusion:

This task provided practical knowledge of Python file handling. I successfully practiced creating, writing, reading, and appending data in .txt files and worked with CSV files using the built-in csv module.

The task improved my understanding of data storage and retrieval in Python and provided a foundation for working with datasets in future data analysis tasks.